C# Advanced Interview Questions

Senior / Mid-Level .NET Developer Edition

1. Memory Model, Stack & Heap Internals

Q: Explain the CLR memory model — stack vs heap in detail.

The stack is a per-thread LIFO structure for local variables, method parameters, and return addresses. It is fast, fixed in size, and automatically reclaimed when a method returns. The heap is a shared, garbage-collected region for reference-type objects and boxed value types. Large Object Heap (LOH) holds objects >= 85,000 bytes and is collected less frequently.

Q: What are the GC generations and how does promotion work?

Objects start in Gen 0. A GC pass that an object survives promotes it to Gen 1, then Gen 2. Gen 0 collections are frequent and cheap. Gen 2 ('full GC') is expensive. The LOH is part of Gen 2. The GC uses a mark-and-sweep-compact algorithm for Gen 0/1 and only sweeps the LOH (fragmentation risk).

```
// Force GC — avoid in production: GC.Collect(); GC.WaitForPendingFinalizers(); // Check generation of an object: int gen = GC.GetGeneration(myObj); // 0, 1, or 2
```

Note: Calling GC.Collect() in production harms performance. Let the runtime manage collection.

Q: What is the Large Object Heap (LOH) and why does it matter?

The LOH holds objects >= 85 KB (e.g., large arrays). Unlike SOH, the LOH is not compacted by default (compaction is opt-in from .NET 4.5.1). This causes heap fragmentation over time. Mitigation: use ArrayPool to rent and return large arrays instead of allocating them.

```
// Use ArrayPool to avoid LOH allocations: byte[] buffer = ArrayPool.Shared.Rent(1024 * 100); try { /* use buffer */ } finally { ArrayPool.Shared.Return(buffer); }
```

Q: What is Span and Memory? Why are they important?

Span is a ref struct that provides a type-safe, bounds-checked view over contiguous memory (array, stack-allocated, or unmanaged) without allocation. Memory is the heap-compatible version (can be stored in fields, used in async). They are critical for high-performance, zero-allocation code.

```
// Slice without allocation: ReadOnlySpan full = "Hello, World".AsSpan(); ReadOnlySpan hello = full.Slice(0, 5); // no allocation // Stack allocation: Span stack = stackalloc int[128];
```

2. Advanced Generics & Type System

Q: What is covariance and contravariance in generics?

Covariance (out): a generic type can be used as a more derived type. Contravariance (in): can be used as a less derived type. Only supported on interfaces and delegates, and only for reference types.

```
// Covariant: IEnumerable assignable to IEnumerable IEnumerable dogs = new List(); IEnumerable animals = dogs; // OK — IEnumerable // Contravariant: Action assignable to Action Action act = a => Console.WriteLine(a); Action dogAct = act; // OK — Action
```

Q: What is the difference between open and closed generic types?

An open generic type has unresolved type parameters (List). A closed generic type has all parameters specified (List). The CLR creates one JIT-compiled implementation per closed value-type instantiation, but shares one implementation for all reference-type instantiations.

Q: What are generic type constraints and when do you use each?

where T : class — reference type only. where T : struct — value type only (no Nullable). where T : new() — must have parameterless constructor. where T : SomeBase — must inherit SomeBase. where T : ISomeInterface — must implement interface. where T : unmanaged — unmanaged value type (useful for unsafe/interop code). Multiple constraints can be combined.

```
public T CreateAndInit(int value) where T : IInitializable, new() { T obj = new T(); obj.Init(value); return obj; }
```

Q: What is the 'default' keyword in generics?

default(T) returns the zero value for T: null for reference types, 0/false/empty struct for value types. In C# 7.1+, you can write just 'default' and the compiler infers the type.

```
T val = default; // null for class, 0 for int, false for bool, etc.
```

3. Concurrency, Threading & async/await Internals

Q: How does async/await work internally (state machine)?

The C# compiler transforms async methods into a state machine struct. Each await point becomes a state. When the awaited task completes, the state machine resumes from the correct state. If the task is already complete, execution continues synchronously — no thread switch. This is why async has near-zero overhead on already-completed tasks.

Q: What is ConfigureAwait(false) and when should you use it?

ConfigureAwait(false) tells the awaiter not to capture the current SynchronizationContext. This avoids marshalling the continuation back to the original context (e.g., UI thread or ASP.NET request context). Use it in library code to avoid deadlocks and improve performance. In application code (UI, controllers), omit it if you need to update UI or access HttpContext after the await.

```
// Library code — always use ConfigureAwait(false): public async Task FetchAsync(string url) { var data = await httpClient.GetStringAsync(url).ConfigureAwait(false); return data.ToUpper(); // safe — no context needed }
```

Q: What is ValueTask and when should you use it over Task?

ValueTask is a struct that avoids heap allocation when a result is available synchronously (common in caching, hot paths). Task always allocates. Use ValueTask only when profiling shows allocation pressure from Task; otherwise Task is simpler and safer.

```
public ValueTask GetCachedAsync(int key) { if (_cache.TryGetValue(key, out int v)) return new ValueTask(v); // no allocation return new ValueTask(FetchFromDbAsync(key)); // async path }
```

Q: Explain lock, Monitor, Mutex, SemaphoreSlim — when to use each?

lock(obj): syntactic sugar for Monitor.Enter/Exit; fast, thread-local, not async-safe. Monitor: same as lock but with TryEnter timeout. Mutex: OS-level, works across processes, heavy. SemaphoreSlim: allows N concurrent threads; has async-friendly WaitAsync() — preferred for async code over lock.

```
// Async-safe rate limiting: SemaphoreSlim _sem = new SemaphoreSlim(1, 1); async Task SafeWriteAsync() { await _sem.WaitAsync(); try { /* critical section */ } finally {
```

```
_sem.Release(); } }
```

Q: What is the difference between Interlocked and volatile?

volatile ensures the field is read/written directly from memory (no CPU register caching), preventing stale reads in simple flag scenarios. Interlocked provides atomic operations (Increment, Decrement, CompareExchange) which are both atomic and create full memory barriers. Use Interlocked for counters; volatile for simple boolean flags.

Q: What are channels (System.Threading.Channels) and when to use them?

Channels are high-performance, async-friendly producer-consumer queues replacing ConcurrentQueue + SemaphoreSlim patterns. BoundedChannel limits capacity (backpressure). UnboundedChannel has no limit. Ideal for pipelines, event streaming, and background processing.

```
var channel = Channel.CreateBounded(100); // Producer: await
channel.Writer.WriteAsync(42); // Consumer: await foreach (var item in
channel.Reader.ReadAllAsync()) { /* process */ }
```

4. Reflection, Attributes & Expression Trees

Q: What is reflection and what are its performance implications?

Reflection (System.Reflection) allows runtime inspection and invocation of types, methods, and properties. It is powerful but slow (~50-100x slower than direct calls) due to metadata lookup and lack of JIT optimization. Cache reflected MemberInfo objects. For hot paths, use compiled delegates or source generators instead.

```
// Cache and compile for performance: var prop = typeof(MyClass).GetProperty("Name");
var getter = (Func)Delegate.CreateDelegate( typeof(Func), prop.GetGetMethod()); //
getter is now a compiled delegate — fast!
```

Q: What are Expression Trees and how are they used?

Expression Trees (System.Linq.Expressions) represent code as data — an AST you can inspect, transform, or compile. Entity Framework uses them to translate LINQ queries to SQL. You can build dynamic queries, mappers, and validators without runtime reflection overhead.

```
// Build a compiled getter at startup: var param = Expression.Parameter(typeof(Person),
"p"); var body = Expression.Property(param, "Name"); var lambda =
Expression.Lambda>(body, param); var getter = lambda.Compile(); // very fast after
compilation string name = getter(person);
```

Q: What are custom attributes and how do you create them?

Attributes are metadata attached to types, methods, or properties, readable at runtime via reflection. Inherit from System.Attribute. Use AttributeUsage to restrict targets. Common in validation frameworks, ORMs, and serializers.

```
[AttributeUsage(AttributeTargets.Property)] public class MaxLengthAttribute : Attribute
{ public int Length { get; } public MaxLengthAttribute(int length) => Length = length;
} // Usage: public class User { [MaxLength(50)] public string Name { get; set; } }
```

5. Design Patterns in C#

Q: Implement the Singleton pattern thread-safely in C#.

The preferred approach uses Lazy which is thread-safe and lazy by default without explicit locking.

```
public sealed class Singleton { private static readonly Lazy _instance = new Lazy(() => new Singleton()); private Singleton() { } public static Singleton Instance => _instance.Value; }
```

Q: What is the Repository pattern and Unit of Work?

Repository abstracts data access behind an interface — the domain layer doesn't know about EF or SQL. Unit of Work groups multiple repository operations into a single transaction. In EF Core, DbContext already implements both patterns, so a thin wrapper is often sufficient.

Q: What is the Strategy pattern? Give a C# example.

Strategy defines a family of algorithms, encapsulates each, and makes them interchangeable. Useful when you want to switch behavior at runtime without conditionals.

```
public interface ISortStrategy { void Sort(List data); } public class QuickSort : ISortStrategy { public void Sort(List d) { /* ... */ } } public class MergeSort : ISortStrategy { public void Sort(List d) { /* ... */ } } public class Sorter { private ISortStrategy _strategy; public Sorter(ISortStrategy s) => _strategy = s; public void Sort(List d) => _strategy.Sort(d); }
```

Q: What is the Decorator pattern vs Inheritance?

Decorator wraps an object to add behavior dynamically, favouring composition over inheritance. It avoids deep inheritance hierarchies. In .NET, Stream decorators (GZipStream, CryptoStream wrapping FileStream) are classic examples.

Q: Explain the Observer pattern vs C# events.

The Observer pattern defines a one-to-many dependency so that when one object changes state, all dependents are notified. C# events are a built-in implementation of this pattern with type safety, += / -= subscription syntax, and encapsulation of the underlying delegate.

6. SOLID Principles & Clean Code

Q: Explain each SOLID principle with a C# context.

S — Single Responsibility: a class should have one reason to change (e.g., OrderService handles order logic; separate EmailService handles notifications). **O** — Open/Closed: open for extension, closed for modification (add new behaviour via new classes, not modifying existing). **L** — Liskov Substitution: subtypes must be usable wherever base types are (don't throw NotImplementedException in overrides). **I** — Interface Segregation: many small focused interfaces over one fat one. **D** — Dependency Inversion: depend on abstractions (interfaces), not concretions — enabled by DI containers.

Q: What is the difference between composition and inheritance? Which is preferred?

Inheritance (is-a) creates tight coupling and fragile hierarchies. Composition (has-a) assembles behaviour from smaller pieces and is more flexible. The general rule: prefer composition over inheritance. Use inheritance for true is-a relationships (e.g., SqlException : DbException).

7. Advanced LINQ & IQueryable Internals

Q: How does LINQ to SQL (IQueryable) differ from LINQ to Objects (IEnumerable)?

IEnumerable processes data in memory — the full dataset is fetched first, then filtered. IQueryable builds an expression tree that a provider (EF Core) translates to SQL — filtering happens in the database. Always use IQueryable with EF, and call .AsEnumerable() only when you need in-memory operations.

Q: What is the N+1 query problem in EF Core and how do you fix it?

N+1 occurs when you load a list (1 query) then access a navigation property in a loop, causing N additional queries. Fix: use Include() for eager loading, or use a projection with Select() to fetch only what you need.

```
// BAD — N+1: var orders = context.Orders.ToList(); foreach (var o in orders) Console.WriteLine(o.Customer.Name); // N queries! // GOOD — eager load: var orders = context.Orders.Include(o => o.Customer).ToList();
```

Q: What is the difference between Select and SelectMany in LINQ?

Select projects each element to a new form (one-to-one). SelectMany flattens one-to-many: projects each element to a sequence and merges all sequences into one flat result.

```
var tags = posts.SelectMany(p => p.Tags).Distinct(); // all tags across all posts
```

8. C# Source Generators & Compile-Time Features

Q: What are Source Generators and why are they significant?

Source Generators (Roslyn, C# 9+) run during compilation to generate additional C# source code. They eliminate reflection-based patterns (serializers, mappers, DI registration) by generating code at compile time — improving startup time and AOT/trimming compatibility. System.Text.Json uses them for zero-reflection serialization.

Q: What is the difference between Source Generators and T4 templates?

T4 runs as a pre-build step and produces static output. Source Generators are Roslyn-integrated, incremental, and participate in IDE features (completions, errors). They are more robust, faster, and support IDE tooling.

9. Unsafe Code, Interop & Performance

Q: What is unsafe code in C#? When is it justified?

The 'unsafe' keyword allows pointer arithmetic and direct memory manipulation, bypassing CLR safety. Justified for P/Invoke interop with native code, writing high-performance algorithms (image processing, parsers), or interfacing with hardware. Must be enabled in project settings.

```
unsafe void CopyMemory(byte* src, byte* dst, int count) { for (int i = 0; i < count; i++) dst[i] = src[i]; }
```

Q: What is P/Invoke and how do you call a native DLL?

Platform Invocation Services (P/Invoke) lets managed code call unmanaged functions in native DLLs. Use DllImport attribute to declare the signature. The runtime marshals types between managed and unmanaged representations.

```
[DllImport("user32.dll", CharSet = CharSet.Unicode)] public static extern int MessageBox(IntPtr hWnd, string text, string caption, uint type);
```

Q: What is stackalloc and when is it useful?

stackalloc allocates a block of memory on the stack (not heap), avoiding GC pressure. Used with Span for temporary buffers. The memory is freed automatically when the method returns. Limit to small, fixed-size buffers (a few KB); the stack is small (~1 MB default).

```
Span buffer = stackalloc byte[256]; // Use buffer — no heap allocation, no GC involvement
```

10. Advanced .NET Topics: DI, Middleware & EF Core

Q: What is the Scrutor library and when would you use it?

Scrutor adds assembly scanning to the built-in .NET DI container, enabling automatic registration of services by convention (e.g., register all classes implementing IService). Reduces boilerplate in large projects.

Q: How do you implement a custom middleware in ASP.NET Core?

A middleware is a class with an Invoke or InvokeAsync method that receives HttpContext and the next RequestDelegate. Register it with app.UseMiddleware() or app.Use().

```
public class TimingMiddleware { private readonly RequestDelegate _next; public TimingMiddleware(RequestDelegate next) => _next = next; public async Task InvokeAsync(HttpContext ctx) { var sw = Stopwatch.StartNew(); await _next(ctx); Console.WriteLine($"Request took {sw.ElapsedMilliseconds}ms"); } }
```

Q: What is EF Core's change tracker and how does AsNoTracking help?

EF Core's change tracker monitors all loaded entities for mutations and generates SQL on SaveChanges(). For read-only queries, AsNoTracking() skips tracking, reducing memory usage and improving performance by ~30% for large result sets.

```
// Read-only query — skip tracking: var users = await context.Users .AsNoTracking() .Where(u => u.IsActive) .ToListAsync();
```

Q: What are compiled queries in EF Core?

EF Core must translate LINQ to SQL on each call (query compilation cost). EF.CompileAsyncQuery caches this translation, eliminating re-compilation overhead on hot paths.

```
private static readonly Func> GetUserById = EF.CompileAsyncQuery((AppDbContext db, int id) => db.Users.FirstOrDefault(u => u.Id == id)); // Usage: var user = await GetUserById(context, 42);
```

11. Common Advanced Interview Traps & Edge Cases

Struct mutability in collections: Structs in List are copies. Modifying list[i].X does nothing — you must copy out, modify, and copy back. This is a common subtle bug.

Closure over loop variables: Capturing a loop variable in a lambda captures the reference, not the value. By the time the lambda runs, the loop may have finished. Fix: copy to a local variable inside the loop.

IDisposable in async: await inside a using block is safe in C# — the compiler ensures Dispose is called after the async method resumes. But mixing sync Dispose with async work can cause issues.

String interning: String literals are interned — same value shares one object. Runtime strings are not interned by default. Don't rely on ReferenceEquals for string equality.

Task.WhenAll vs Task.WhenAny: WhenAll awaits all tasks (throws AggregateException if any fail). WhenAny returns as soon as the first task completes. Forgetting to await remaining tasks from WhenAny causes unobserved exceptions.

Lazy exception caching: If the factory in Lazy throws, the exception is cached and re-thrown on every subsequent access (default mode). Use LazyThreadSafetyMode.PublicationOnly to retry on failure.

Enum flags and bitwise ops: For [Flags] enums, use HasFlag() or bitwise & for checking. Assigning with | combines flags; & ~ removes a flag. Without [Flags], these operations have no semantic meaning.
